Worklive Memory and Office Growth

Product requirements and memory system research

PRD addendum 1.3 • 30 September 2026

Recommendation. Give every agent a persistent identity and private notebook, and give every project a shared, timestamped knowledge base. Every agent can use either Codex or Claude Code while retaining the same authorized memories. Store memory locally in Worklive. Start evaluation with SQLite and full-text retrieval; release only after memory quality passes on both providers, adding local semantic retrieval if needed.

The office should expand through small rooms, department floors and, eventually, several buildings. Agent count determines available space. Subscription capacity determines how many agents can actually work at once. These are separate product controls.

This addendum extends the existing Worklive PRD. It replaces the earlier always-visible name labels with compact badges and details on hover or keyboard focus. It specifies memory, growth and customization for upcoming releases; those systems are not implemented by the label update.

Decisions to carry forward

Area	Proposed decision
Agent memory	Private notes and reusable lessons belong to a stable agent ID, independent of name, provider and appearance.
Project memory	Members can read and append findings; every entry names its author, source, time and status. Canonical decisions have explicit owners.
Execution	Every agent supports either official CLI. Memory survives provider changes; extraction and reflection use the selected CLI, with no hidden model API or hosted embedding fallback.
Office growth	Rooms contain up to 12 desks; four rooms form a floor and four floors form a building. These are proposed layout units to prototype.
Character identity	Personality, role and appearance are customizable. Codex teal and Claude terracotta remain reserved for the visible shirt.

What changes now

Worklive 0.3.1 replaces floating information cards with small initial badges. Hovering a character or badge reveals its name, provider, role and activity; keyboard focus reveals the same details. Clicking keeps full information in the side panel. The current six-agent limit and two-task execution limit remain in place.

1 Memory systems compared

There is no universal winner. The strongest choice depends on whether the product needs personalization, shared working context, temporal relationships, or local retrieval. This is a documentation review of current primary sources, not a measured performance ranking.

System	Useful capability	Fit for Worklive
Letta [S1]	Bounded memory blocks can be shared between agents; some blocks can be read-only.	Best reference for private and shared notebooks. Replacing the CLI runtime with Letta is unnecessary for the next milestone.
Mem0 [S2]	Agent and user memory, extraction and retrieval. Current docs describe append-only extraction and several retrieval signals.	A useful general memory layer, but defaults include a model and embedding calls. Its managed benchmark results are explicitly not identical to the open-source SDK.
Graphiti and Zep [S3]	Temporal knowledge graphs for evolving facts and relationships.	Strong candidate if dependency and change-over-time questions dominate. The graph service, extraction and embeddings add operational work beyond a local notebook.
Hindsight [S4] [S5]	Memory banks organize retention, retrieval and reflection.	Most interesting advanced-memory candidate for a later prototype. Bank boundaries align with agents and projects, but model transport and background jobs need close control.

The subscription integration trap

“Works with Codex” can mean that Codex calls a memory tool, or that the memory service itself uses a model. Both directions need to be inspected. A tool integration alone does not establish that its internal extraction and embedding steps use the official CLI.

Hindsight’s current documentation describes its Claude Code backend as using the official Agent SDK and bundled CLI, with a local personal-development qualification. Its openai-codex backend reads Codex authentication data; its configuration identifies an HTTP transport. That second path does not meet Worklive’s CLI-only design. We should not enable it as a shortcut. [S5] [S12]

Self-hosting storage does not by itself remove model calls. Worklive should retain ownership of scheduling, provider choice, cancellation and usage visibility for any memory processing.

2 Recommended local foundation

Use SQLite as the authoritative memory store, with FTS5 keyword search as the first retrieval baseline. SQLite provides full-text indexing and BM25 ranking locally. Our recommendation is based on integration fit and inspectability, not a claim that keyword search matches semantic retrieval quality. Quality gates determine what ships. [S9]

This gives us transactions, explicit ownership, revisions and exportable records without introducing another model service. The agent can formulate memory searches and structured notes through its existing CLI session. Storage operations themselves require no inference.

Candidate	Role in the design	Decision
LangGraph [S6]	Separates thread state from long-term memory organized by namespaces.	Borrow this separation. Worklive can implement it without adopting the whole orchestration framework.
AgentDB [S7]	Local agent memory with episodes, reusable skills, a Node API and MCP tools.	Shortlist for a measured local-memory spike. Audit the pinned version and expose only tools within Worklive's scope.
RuVector [S8]	Rust vector and agent-memory components with documented local embedding options.	Shortlist for a Rust integration if semantic retrieval is justified. Verify persistence, deletion, isolation and recovery in the actual adapter.
Qdrant [S10]	Vector storage with payload filtering and multitenancy options.	Consider for a future shared service. It is a retrieval engine, not a complete policy or memory lifecycle.

A replaceable search layer

Keep a stable MemoryStore boundary with search, read, append, supersede and forget operations. Rust owns membership checks and canonical records. A derived vector index can be rebuilt from those records. It must not become the only place where permissions, decisions or provenance exist.

If keyword retrieval misses required evidence, evaluate local embeddings and hybrid search before release. Use exact vector search as the baseline before adding an approximate index. A local embedding model is an explicit component, not another API credential; hashed vectors are not a substitute for semantic retrieval.

Why not pick from leaderboard numbers

Vendor benchmarks can use different corpora, model backbones, retrieval budgets and proprietary services. Mem0 explicitly distinguishes its platform results from its open-source SDK. Worklive needs its own tests for shared projects, corrections, permissions and provider switching. [S2]

3 Personal and project memory

Use four scopes with explicit access rules. The desktop owner can inspect, correct, export and forget stored memories. “Private” means hidden from other agents; it does not mean hidden from the user.

Scope	Content	Who can use it
Agent identity	Name, role, character style and working preferences.	The owner and that agent. Identity is configuration, not evidence that fictional events occurred.
Agent notebook	Task lessons, personal working notes and reusable methods.	That agent and the owner. Notes derived from a project retain the source project’s access restrictions.
Project knowledge	Brief, accepted decisions, findings, references, open questions and handoffs.	Current project members according to read or contribute permission. Sharing a floor does not grant access.
Run evidence	Observed sources, outputs, tool events and the memory context used during one run.	The run’s authorized participants and owner. This supports audit and replay without treating every log line as durable knowledge.

Required record fields

Every memory has a stable ID, workspace ID, scope and scope ID, kind, content, author agent or user ID, source run ID, source references and content hash. Project-derived records also carry their originating project and any access restrictions that must follow them.

Store **recorded_at** for when Worklive saved the entry, **observed_at** for when the evidence was seen, and optional **valid_from** and **valid_to** for the period the statement describes. Persist UTC; display the user’s local time with the original time available.

Include verification state, reviewer if any, revision, supersedes ID, optional expiry and an idempotency key. A model’s confidence is a claim, not verification. The host stamps author identity, provider, timestamps and scope from the active run; the agent cannot impersonate another author.

A useful memory is small and attributable

Save “The approved export format is Markdown, decided by the owner on 30 September” with its decision reference, rather than a whole conversation labeled “project facts.” Preserve the supporting artifact separately. Do not store passwords, session tokens or casual coffee chatter as project knowledge.

4 Changes contradictions and forgetting

Concurrent collaboration

Members may append observations, lessons, proposals and handoff notes immediately. Updating an existing canonical item uses an expected revision in a transaction. A stale writer receives a conflict and must read the new state; the most recent timestamp does not automatically make a claim true.

Project decisions and pinned instructions have a designated owner. Agents can propose a replacement, but cannot quietly replace an owner-approved rule. Every superseding decision links back to the earlier version. Findings remain distinguishable from proposals and verified decisions.

Illustrative project timeline, not actual Worklive history:

Recorded time UTC	Entry	Current treatment
30 Sep 09:00	Owner approves Markdown as the first export format.	Accepted decision.
30 Sep 09:08	Ada proposes adding CSV for comparisons.	Proposal; does not override the decision.
30 Sep 09:15	Owner approves Markdown plus CSV, superseding the earlier decision.	Current decision; earlier version remains available in history.

Keeping current knowledge current

Retrieval normally excludes expired, deleted and superseded entries. A historical question can request an explicit time range. Unresolved contradictions remain visible with both sources. Important decisions can be pinned; weak evidence must not win merely because several agents repeated it.

Forgetting and revocation

Forgetting removes content from active records, text and vector indexes, derived summaries and cached context. Retain only a content-free deletion event where audit is needed. Regenerate summaries that depended on the entry; do not keep the forgotten text inside an “audit” copy.

Removing project membership stops new reads and cancels runs using that project context. A new run gets fresh, authorized context. Data already sent to a provider cannot be recalled from its context, and CLI transcripts and external backups require their own retention controls. The UI must explain the scope of deletion accurately.

An agent’s cross-project notebook must not become a route around project membership. Reusable, sanitized lessons can be promoted to a broader scope by an explicit user action; confidential source material keeps its original restrictions.

5 Memory in the real work loop

Integration with the existing CLIs

At task start, Worklive binds the stable agent ID, chosen provider, project membership and permissions to the run. It selects current project decisions, a small project summary and relevant allowed notebook entries, then records the IDs and revisions included in the task context.

Every agent supports both providers through the same memory contract. Switching starts a fresh session with the chosen CLI and the same authorized memories; it does not copy a provider's private transcript or reuse its session ID. New entries record which provider authored them without making access depend on that provider.

Expose narrow memory tools through the existing local MCP bridge: memory_search, memory_read, memory_append and memory_propose_update. The bridge derives identity and permitted scopes from the run. It never accepts a claimed author or an arbitrary workspace path as authorization.

Agents write concise memory entries during useful work. Optional end-of-task consolidation is a visible, bounded CLI task using the selected provider and the same scheduling limits. Idle coffee animations do not launch reflection calls. Never make invisible per-message model calls in a memory service.

The host must enforce permissions before returning content and before writes. Memory text remains source material: an entry cannot grant browser, filesystem or other privileges. For a membership or project change, start a fresh CLI session instead of resuming one that still contains inaccessible context.

The notebook interface

Add Personal memory and Project memory tabs to the agent panel. Show searchable entries with author, timestamps, source, status and revision history. Provide Pin, Correct, Move through an explicit sharing action, Export and Forget. A task's work panel shows "Context used" and "Memories added" so the user can inspect the effect of memory.

Retrieval evaluation before choosing a vector engine

Build a held-out set of 200 labeled questions spanning preferences, project facts, temporal changes, contradictions and deliberate no-answer cases. Run it against 10,000 and 100,000 records across up to 1,000 agents and 100 projects. These are proposed test sizes, not measurements already obtained.

Report evidence recall@8, grounded answer correctness, temporal correctness, p50/p95 retrieval time, ingestion and rebuild time, memory use, disk use and prompt size. Compare FTS with exact vector search and candidate indexes using the same embeddings. Score Codex and Claude Code separately; an average must not hide one provider's failures.

Proposed release gates: at least 90% evidence recall@8 and 90% grounded answer correctness on answerable cases for each provider; at least 95% correct abstention when evidence is absent; and zero unauthorized content. Critical corrections and current approved decisions must pass. Performance targets are p95 retrieval below 150 ms at 100,000 records and about 1,500 context tokens. Validate on declared hardware; these are targets, not measured claims.

6 An office that grows

Use a stable hierarchy: campus → building → floor → room → desk → agent. Departments influence placement; project membership controls knowledge access. A person can work across departments and projects without changing where their desk lives.

Agents	Desk rooms	Floors	Buildings	Default view
1–12	1	1	1	One cozy room with individual characters.
20	2	1	1	Current room plus another room and shared amenities.
48	4	1	1	Floor overview with a room selector.
100	9	3	1	Department and floor overview; enter one room.
1,000	84	21	6	Campus overview and searchable people directory.

Counts illustrate dense packing at 12 desks per room, four rooms per floor and four floors per building. Department separation and reserved amenity space may require more rooms. They are proposed capacity rules, not a claim that the current renderer supports these counts.

Growth without rearranging everybody

Place a new agent into a free desk in its department. If needed, attach a room, then open a floor, then add a building. Keep existing desk IDs and positions stable. Announce expansion gently and preserve the user’s camera. Archived agents vacate desks, but do not automatically demolish a floor or remove a lounge. Allow manual moves and department reassignment.

A thousand agents does not mean a thousand browsers

Persist all agent profiles, but render detailed bodies and animation in the current room only. Floor and campus views use counts, simple activity markers and department summaries. Use shared meshes for repeated furniture and reduced detail at distance; Three.js provides instancing primitives for that purpose. [S11]

Create browser contexts when an agent starts work or its computer is opened. Keep a bounded cache of paused contexts and a retained last frame for sleeping desks, visibly labeled with its age. Detailed collision simulation runs only in the visible room; offscreen agents use inexpensive schedule state.

A separate queue limits running CLIs and browser workers. Preserve today’s one task per provider until capacity is measured and user controls are available. At scale, search and filters must reach any agent, department, project, blocked task or finished result without navigating every floor.

7 Character styles and office rewards

Separate personality from provider identity

Store role, persona, appearance and provider as independent fields. Role defines the work specialty; persona shapes tone and habits; appearance selects body, hair and accessories; provider chooses the executing CLI and the shirt palette. Names and providers can change without replacing the agent ID or its notebook.

A Gilfoyle-inspired preset could be a skeptical infrastructure specialist with dry humor, concise explanations, dark hair, a beard and restrained gestures. An original fantasy preset could be a patient archivist with pointed ears, glasses, a hat and a small familiar. Users can also describe their own characters.

Personality should shape presentation without inventing completed work or professional experience. Fictional background belongs in the character profile, separate from factual project memory. Presets are editable starting points, not fixed competence or permission levels.

Provider colors are invariant

Codex remains teal, #2D9D8F; Claude Code remains terracotta, #D97757. The visible torso and sleeves use the provider material. Hats, hair, glasses, boots, props and desk decorations can vary. A robe, jacket or armor must keep the provider-colored shirt visibly exposed or incorporate that same material into its dominant torso and sleeves. A tiny hidden badge is insufficient.

Keep a provider name and distinct badge shape in the detail UI for people who cannot distinguish the colors. If the user changes providers during a run, keep the executing provider’s shirt until it finishes; apply the new provider to the next run.

Unlock one amenity at each step

Roster milestone	New space or prop	Local activity
10 agents	Chill lounge	Sit, read or have an informal conversation.
20 agents	Table tennis	Short paired breaks.
30 agents	TV area	Watch together during a break.
40 agents	Gaming area	Play a short game together.
50 agents	Balcony or garden	Walk outside and water plants.
60 agents	Rooftop terrace	Small gatherings and celebrations.
70 agents	Courtyard	Walk and meet outdoors.

Unlock one amenity per milestone. The 20, 30 and 40 steps are fixed requirements; 50 onward are proposed extensions. Count active roster members, not concurrent runs or tokens. Earned amenities remain available after archiving. Creative mode allows free placement. Breaks never gate work or start model calls by default.

8 Delivery plan and acceptance

Stage	Deliverable	Acceptance
Now 0.3.1	Compact office badges and details on hover or focus.	Full cards no longer cover the scene. Selection, monitors, keyboard access and provider colors still work.
Next Memory foundation	Local notebooks, projects, scoped MCP tools and quality evaluation on both CLIs.	Either provider recalls corrections and shared findings. Both pass quality gates; nonmembers cannot access project memory.
Then Collaboration	Explicit handoffs, accepted project decisions, version conflicts and context inspection.	Two agents can contribute without silently overwriting each other. Removing access revokes active work and requires fresh context.
Then Office growth	12-desk rooms, floor navigation, directory and lazy browser sessions.	A synthetic 100-agent roster remains usable without starting 100 browsers. Stable desks survive growth and restart.
Later Campus and styles	Buildings, amenities, character editor and optional semantic retrieval.	1,000-profile load testing, bounded resource use and proven retrieval improvements precede a support claim.

Memory release checks

- Preserve memory through renames and restart. Test Codex to Codex, Claude to Claude and switches in both directions: write, correct and retrieve the same authorized agent and project memories. A provider change starts a fresh CLI session.
- Test unauthorized searches, direct reads, writes and guessed IDs across agents, projects and workspaces. Filters in prompts are not a substitute for host checks.
- Test two writers editing the same revision, replayed writes, duplicate IDs, invalid timestamps, malformed notes, disk-full errors and interrupted transactions.
- Verify a superseded deadline is not returned as current, deleted content disappears from every derived index and summary, and source excerpts cannot turn into instructions.
- Measure cold start, indexing, recovery and retrieval on declared hardware before promoting any memory adapter.

Visual release checks

Check label occlusion in crowded rooms, keyboard focus, hover details at viewport edges, long names, reduced motion and every provider/style combination. Maintain visible provider color during walking, sitting and prop use. Profile editing must never overwrite provider materials.

For growth, measure visible-room frame times, memory use and floor-switch latency at synthetic rosters of 100 and 1,000. Offscreen agents must not create independent render loops, browser captures or idle inference. Character and amenity work follows this resource model rather than hiding scaling costs behind more scenery.

9 Sources and interpretation

Primary documentation reviewed on 30 September 2026. Linked references support system capabilities and integration constraints. Layout capacities, milestones, memory policies and evaluation thresholds are Worklive design proposals. No package was installed, no credentials were read, and no vendor performance benchmark was reproduced for this review.

S1 Letta memory blocks

Shared blocks and bounded context.

S2 Mem0 repository

Current memory pipeline, embedding defaults and benchmark qualifications.

S3 Graphiti repository

Temporal graph memory and deployment requirements.

S4 Hindsight memory banks

Choosing boundaries for isolated banks.

S5 Hindsight model integration

Extraction and reflection models; distinct Claude Code and Codex paths.

S6 LangGraph memory overview

Conversation state versus long-term namespaced memory.

S7 AgentDB repository

Episodic memory, skills and local integration surfaces.

S8 RuVector repository

Rust memory components, local embeddings and namespace mechanisms.

S9 SQLite FTS5

Local full-text indexing and BM25 ranking.

S10 Qdrant multitenancy

Filtered vector collections and tenant partitioning.

S11 Three.js InstancedMesh

Shared geometry and materials for repeated objects.

S12 Hindsight configuration

Model transports and embedding configuration.